The AI Operating System — Part I

Agent Scheduler: Complex Orchestration as Process Management

A Categorical Framework for Agent Lifecycle, Supervision, and Durable Execution

Matthew Long
The YonedaAI Collaboration
YonedaAI Research Collective
Chicago, IL
matthew@yonedaai.com · https://yonedaai.com

March 10, 2026

Abstract

We introduce a formal framework for understanding AI agent scheduling as the fundamental primitive of an AI Operating System (AIOS). Drawing on classical operating systems theory, category theory, and the Erlang/OTP supervision model, we show that complex agent orchestrations — web testing pipelines, legal analysis workflows, security auditing systems — can be modeled as objects in a category whose morphisms encode data flow and compositional structure. We define an *execution category* whose objects are agent states and whose morphisms are state transitions, and prove that durable execution engines (modeled on Inngest) preserve idempotency as a functorial invariant. We formalize the Agent-Hero execution pipeline (Job → Proposal → Contract → Decomposition → Execution → Evaluation) as a chain complex in a preadditive category and demonstrate that supervision trees correspond to limit-preserving functors from the category of failure modes to the category of recovery strategies. We show that oversight modes (supervised, spot-check, autonomous escalation) form a complete lattice under a refinement ordering, and that the 6-dimensional quality evaluation (quality, adherence, speed, cost, error, revision) defines a weighted inner product space suitable for agent reputation computation. We validate our framework through three case studies drawn from production systems and provide a complete Elixir/OTP reference implementation demonstrating supervision trees, streaming pipelines, and fault-tolerant scheduling.

Keywords: AI operating systems, agent scheduling, category theory, OTP supervision, durable execution, streaming pipelines, formal verification

Contents

1 Introduction **2**

2 Background: Traditional Process Scheduling **3**
2.1 The Linux Completely Fair Scheduler . . . 3
2.2 Priority Queues and Preemption . . . 3
2.3 Process States and Lifecycle . . . 3
2.4 Supervision in Traditional Systems . . . 3
2.5 From Processes to Agents . . . 4

3 Agent Model: Categorical Foundations **4**
3.1 The Agent Category . . . 4
3.2 The Execution Pipeline as a Diagram . . . 5
3.3 Evaluation Metrics and Functoriality . . . 6
3.4 Supervision Trees as Limit-Preserving Functors . . . 6

4 Scheduling Architecture **7**
4.1 Credit-Weighted Priority Scheduling . . . 7
4.2 Dependency Resolution via Topological Sort . . . 8
4.3 Streaming Pipeline Scheduling . . . 9
4.4 Fault Tolerance via Supervision . . . 9

5 Formal Framework: Durable Execution and Oversight **10**
5.1 The Execution Category . . . 10
5.2 Durable Execution and Idempotency . . . 11
5.3 Oversight Modes as a Lattice . . . 12
5.4 Executor/Evaluator/Orchestrator Separation . . . 13

6 Elixir/OTP Implementation **13**
6.1 Application Supervision Tree . . . 13
6.2 Agent GenServer . . . 14
6.3 Priority-Based Scheduler . . . 15
6.4 Streaming Pipeline . . . 16
6.5 Dynamic Supervisor for Agent Pools . . . 16
6.6 6-Dimensional Evaluator . . . 17

7 Case Studies **18**
7.1 Case Study 1: Web Testing Agent . . . 18
7.2 Case Study 2: Legal Analysis Agent . . . 19
7.3 Case Study 3: Security Auditing Agent . . . 19

8 Evaluation Metrics **20**
8.1 The 6-Dimensional Quality Space . . . 20
8.2 Reputation Computation . . . 20
8.3 Comparative Evaluation Table . . . 21
8.4 Scheduling Impact on Evaluation . . . 21

9 Discussion **22**
9.1 Comparison to Linux CFS . . . 22
9.2 Comparison to Real-Time Scheduling . . . 22
9.3 Relationship to Multi-Agent Systems . . . 22
9.4 Category Theory in Systems Design . . . 23

9.5 Durable Execution in Practice 23
9.6 Limitations and Future Work 23

10 Conclusion **24**

A Complete Agent State Machine Specification **26**

B Proof: Agent Category Has Products **27**

C Scheduling Algorithm Complexity Analysis **27**

1 Introduction

The emergence of large language model (LLM) agents as autonomous computational entities has created an urgent need for systems that manage their lifecycle, scheduling, fault tolerance, and evaluation with the same rigor that traditional operating systems bring to process management (Wang et al., 2024; Xi et al., 2023). Unlike simple API calls, modern AI agents are *complex orchestrations*: multi-phase workflows that involve planning, tool use, streaming data pipelines, human oversight, and quality evaluation. Companies deploy agents for web testing, legal document analysis, security auditing, dashboard generation, and dozens of other domains — each requiring workflow decomposition, dependency resolution, retry logic, and compositional validation (Wu et al., 2023; Hong et al., 2024).

This paper argues that **agent scheduling is the core primitive of an AI Operating System**, just as process scheduling is the core primitive of Unix/Linux. We develop this analogy rigorously, mapping:

- **Processes** $\mapsto$ **Agents**: autonomous units of computation with lifecycle states
- **The Linux scheduler (CFS)** $\mapsto$ **The Agent Scheduler**: priority-based, credit-weighted allocation of compute, LLM tokens, and tool access
- **Process groups and sessions** $\mapsto$ **Agent teams and contracts**: hierarchical groupings with shared resource budgets
- **Signals and IPC** $\mapsto$ **Event streams and message passing**: the EventEmitter-based streaming bus where downstream agents consume upstream events
- **Supervision (systemd, init)** $\mapsto$ **OTP supervision trees**: Erlang/Elixir's "let it crash" philosophy applied to agent fault tolerance

Our contributions are:

1. A **categorical formalization** of agents as objects in a category **Agent** with morphisms encoding data flow, and a proof that agent composition preserves evaluation metrics (Theorem 3.9).
2. A formal model of **durable execution** showing that Inngest-style idempotent step functions preserve state consistency under retry (Theorem 5.5).
3. A **lattice-theoretic characterization** of oversight modes with precise ordering and meet/join operations (Proposition 5.7).
4. A **6-dimensional evaluation framework** formalized as a weighted inner product space with convergence guarantees for reputation computation (Theorem 8.5).
5. A complete **Elixir/OTP reference implementation** demonstrating production-quality supervision, scheduling, and streaming pipelines.

The rest of this paper is organized as follows. Section 2 reviews traditional OS scheduling. Section 3 develops the categorical agent model. Section 4 presents the scheduling architecture. Section 5 develops the formal framework for durable execution and oversight. Section 6 provides the Elixir/OTP implementation. Section 7 presents case studies. Section 8 formalizes the evaluation metrics. Section 9 compares our approach to existing systems. Section 10 concludes.

2 Background: Traditional Process Scheduling

2.1 The Linux Completely Fair Scheduler

The Linux Completely Fair Scheduler (CFS), introduced in kernel 2.6.23 (Pabla, 2009), replaced the O(1) scheduler with a design based on *virtual runtime* (vruntime). Each process accumulates vruntime proportional to its actual CPU time, weighted inversely by its nice value. The scheduler always selects the process with the smallest vruntime, maintained in a red-black tree for $O(\log n)$ access (Love, 2010).

Definition 2.1 (Virtual Runtime)**.** For a process p with nice value n_p, the virtual runtime after executing for real time Δt is:

$$\text{vruntime}(p) \mathrel{+}= \Delta t \cdot \frac{w_0}{w(n_p)} \tag{1}$$

where $w(n_p) = 1024/1.25^{n_p}$ is the weight corresponding to nice value n_p, and $w_0 = w(0) = 1024$ is the default weight.

The key insight of CFS is that *fairness is defined relative to weight*: a process with twice the weight accumulates vruntime at half the rate, effectively receiving twice the CPU time. This weighted fairness model directly inspires our credit-weighted agent scheduling (Section 4).

2.2 Priority Queues and Preemption

Real-time scheduling in Linux uses two policies: `SCHED_FIFO` (first-in, first-out within priority level) and `SCHED_RR` (round-robin within priority level), both with 99 priority levels (Bovet and Cesati, 2005). Real-time processes always preempt CFS processes. This two-tier structure — real-time above fair-share — maps to our distinction between *contracted agents* (guaranteed SLA) and *marketplace agents* (best-effort scheduling).

2.3 Process States and Lifecycle

A Linux process transitions through states: `TASK_RUNNING`, `TASK_INTERRUPTIBLE`, `TASK_UNINTERRUPTIBL` `TASK_STOPPED`, `TASK_ZOMBIE`, and `EXIT_DEAD` (Bovet and Cesati, 2005). The state machine is driven by system calls, signals, and scheduler decisions. We will define an analogous state machine for agents in Section 3, with states corresponding to the Agent-Hero execution pipeline.

2.4 Supervision in Traditional Systems

The Unix init system (and its modern replacement, systemd) provides process supervision: restarting crashed services, managing dependencies, and enforcing resource limits (Poettering, 2010). However, Unix supervision is coarse-grained — it operates at the service level, not within an application's internal structure. Erlang/OTP's supervision model (Armstrong, 2003) provides a much richer framework, with hierarchical supervision trees, configurable restart strategies (one-for-one, one-for-all, rest-for-one), and explicit separation between worker processes and supervisors. This fine-grained supervision is precisely what agent orchestration requires.

2.5 From Processes to Agents

The analogy between processes and agents is not merely pedagogical — it is structural. Both are autonomous computational entities with:

- A **lifecycle** governed by state transitions
- **Resource requirements** (CPU/memory for processes; tokens/tools/credits for agents)
- **Communication needs** (IPC for processes; event streams for agents)
- **Failure modes** requiring recovery strategies
- **Hierarchical composition** (process groups / agent teams)

The critical difference is that agent execution is *non-deterministic* and *long-running*: a single agent task may take minutes to hours, involve multiple LLM calls with stochastic outputs, require human approval at checkpoints, and produce streaming intermediate results. This demands scheduling algorithms that account for uncertainty, support checkpointing, and integrate evaluation feedback.

3 Agent Model: Categorical Foundations

We now develop a formal categorical framework for agents, their composition, and their supervision.

3.1 The Agent Category

Definition 3.1 (Agent Profile)**.** An *agent profile* is a tuple $A = (I, C, T, O, \sigma, \kappa)$ where:

- I is the *input schema* (a set of typed input parameters)
- C is the *capability set* (tools, models, and APIs the agent can access)
- $T \subseteq \mathcal{T}$ is the *task domain* drawn from a universal task taxonomy $\mathcal{T}$
- O is the *output schema* (a set of typed output artifacts)
- $\sigma \in \{\texttt{supervised}, \texttt{spot_check}, \texttt{autonomous_escalation}\}$ is the *oversight mode*
- $\kappa \in \mathbb{R}_{>0}$ is the *credit cost per invocation*

Definition 3.2 (Agent Category)**.** The *agent category* **Agent** has:

- **Objects:** Agent profiles $A, B, C, \ldots$ as in Definition 3.1
- **Morphisms:** A morphism $f : A \to B$ is a *data flow specification* $(f_{\text{map}}, f_{\text{val}}, f_{\text{meta}})$ where:
  - $f_{\text{map}} : O_A \to I_B$ maps outputs of A to inputs of B
  - $f_{\text{val}} : O_A \to \{0, 1\}$ is a *validation predicate* that must hold before data flows to B

- $f_{\text{meta}} : \mathcal{M}_A \to \mathcal{M}_B$ maps execution metadata (timing, token usage, error counts)

- **Composition:** For $f : A \to B$ and $g : B \to C$, define $g \circ f : A \to C$ by:

$$(g \circ f)_{\text{map}} = g_{\text{map}} \circ f_{\text{map}} \tag{2}$$
$$(g \circ f)_{\text{val}} = f_{\text{val}} \wedge (g_{\text{val}} \circ f_{\text{map}}) \tag{3}$$
$$(g \circ f)_{\text{meta}} = g_{\text{meta}} \circ f_{\text{meta}} \tag{4}$$

- **Identity:** $\text{id}_A = (\text{id}_{O_A}, \text{true}, \text{id}_{\mathcal{M}_A})$

Lemma 3.3. Agent *is a well-defined category: composition is associative and unital.*

Proof. Associativity of composition follows from associativity of function composition for f_{map} and f_{meta}, and from associativity and commutativity of conjunction for f_{val}. The identity morphism satisfies $f \circ \text{id}_A = f$ and $\text{id}_B \circ f = f$ by direct computation: $f_{\text{map}} \circ \text{id}_{O_A} = f_{\text{map}}$, $\text{true} \wedge f_{\text{val}} = f_{\text{val}}$, and similarly for the right identity. □

The following commutative diagram illustrates a three-agent pipeline $A \xrightarrow{f} B \xrightarrow{g} C$:

$$\begin{tikzcd} O_A \arrow[r, "f_{\text{map}}"] \arrow[dr, "(g\circ f)_{\text{map}}"'] & O_B \arrow[d, "g_{\text{map}}"] \\ & O_C \end{tikzcd} \qquad \begin{tikzcd} \mathcal{M}_A \arrow[r, "f_{\text{meta}}"] \arrow[dr, "(g\circ f)_{\text{meta}}"'] & \mathcal{M}_B \arrow[d, "g_{\text{meta}}"] \\ & \mathcal{M}_C \end{tikzcd} \tag{5}$$

3.2 The Execution Pipeline as a Diagram

The Agent-Hero execution pipeline defines a fixed sequence of states:

$$\texttt{Job} \xrightarrow{\alpha} \texttt{Proposal} \xrightarrow{\beta} \texttt{Contract} \xrightarrow{\gamma} \texttt{Decomposition} \xrightarrow{\delta} \texttt{Execution} \xrightarrow{\epsilon} \texttt{Evaluation} \tag{6}$$

Definition 3.4 (Pipeline Category)**.** The *pipeline category* **Pipe** is the free category on the directed graph with vertices $\{J, P, C, D, E, V\}$ (Job, Proposal, Contract, Decomposition, Execution, Evaluation) and edges $\{\alpha, \beta, \gamma, \delta, \epsilon\}$ as in (6). This is equivalent to the ordinal category $\mathbf{6} = \{0 < 1 < 2 < 3 < 4 < 5\}$.

Definition 3.5 (Pipeline Functor)**.** An *agent execution* is a functor $F : \mathbf{Pipe} \to \mathbf{Set}$ mapping each pipeline stage to its state space and each transition to a state transformation function:

$$F(J) = \{\text{job specifications}\} \qquad F(\alpha) : F(J) \to F(P) \tag{7}$$
$$F(P) = \{\text{proposals}\} \qquad F(\beta) : F(P) \to F(C) \tag{8}$$
$$F(C) = \{\text{contracts}\} \qquad F(\gamma) : F(C) \to F(D) \tag{9}$$
$$F(D) = \{\text{task decompositions}\} \qquad F(\delta) : F(D) \to F(E) \tag{10}$$
$$F(E) = \{\text{execution results}\} \qquad F(\epsilon) : F(E) \to F(V) \tag{11}$$
$$F(V) = \{\text{evaluations}\} \tag{12}$$

Remark 3.6. The category of all agent executions is the functor category $[\mathbf{Pipe}, \mathbf{Set}]$, which is a presheaf topos. This gives us access to powerful topos-theoretic tools for reasoning about agent executions, including subobject classifiers for defining "valid" executions and exponential objects for higher-order agent composition. We defer this development to Part III of this series.

3.3 Evaluation Metrics and Functoriality

Definition 3.7 (Evaluation Space)**.** The *evaluation space* is $\mathcal{E} = [0,1]^6$, with coordinates corresponding to the six quality dimensions: quality (q), adherence (a), speed (s), cost efficiency (c), error rate (e), and revision count (r). An *evaluation* of an agent execution is a point $\mathbf{v} = (q, a, s, c, e, r) \in \mathcal{E}$.

Definition 3.8 (Evaluation Functor)**.** The *evaluation functor* Eval : **Agent** $\to$ **Met** maps from the agent category to the category of metric spaces (with short maps as morphisms):

$$\mathrm{Eval}(A) = (\mathcal{E}_A, d_w) \quad \text{where } d_w(\mathbf{v}, \mathbf{v}') = \sqrt{\sum_{i=1}^{6} w_i (v_i - v_i')^2} \tag{13}$$

where $\mathbf{w} = (w_q, w_a, w_s, w_c, w_e, w_r)$ is a weight vector with $\sum w_i = 1$.

Theorem 3.9 (Functorial Composition Preserves Evaluation Bounds)**.** *Let $f : A \to B$ be a morphism in* **Agent** *with validation predicate f_{val}. If Eval$(A) \geq \mathbf{v}_{\min}$ componentwise and f_{val} enforces that each output dimension $o_i \geq \theta_i$ for thresholds θ_i, then:*

$$\mathit{Eval}(B) \geq \Phi_f(\mathbf{v}_{\min}, \boldsymbol{\theta}) \tag{14}$$

where $\Phi_f : \mathcal{E} \times [0,1]^{|O_A|} \to \mathcal{E}$ is a monotone map determined by f_{map} and f_{val}. In particular, Eval preserves the partial order on $\mathcal{E}$ and thus acts as a monotone functor from the poset-enriched agent category to $(\mathcal{E}, \leq)$.

Proof. Let $\mathbf{v}_A = \mathrm{Eval}(A)$ with $\mathbf{v}_A \geq \mathbf{v}_{\min}$. The validation predicate f_{val} acts as a filter: only outputs satisfying $o_i \geq \theta_i$ flow to B. The mapping f_{map} transforms these validated outputs into inputs for B.

Since each dimension of B's evaluation is computed from B's outputs, which depend on the validated inputs from A, we have for each quality dimension i:

$$\mathrm{Eval}(B)_i = g_i(f_{\mathrm{map}}(\mathbf{o}_A)) \tag{15}$$

where g_i is B's quality function for dimension i. Since f_{val} ensures $\mathbf{o}_A$ lies in the feasible region $\{\mathbf{o} : o_j \geq \theta_j \ \forall j\}$, and $g_i \circ f_{\mathrm{map}}$ is continuous on this compact domain, the image is bounded below by $\Phi_f(\mathbf{v}_{\min}, \boldsymbol{\theta})_i = \inf_{\mathbf{o} \in \mathrm{feas}} g_i(f_{\mathrm{map}}(\mathbf{o}))$.

Monotonicity follows: if $\mathbf{v}_{\min} \leq \mathbf{v}'_{\min}$ componentwise, then the feasible region for $\mathbf{v}'_{\min}$ is a subset, and the infimum over a subset is at least as large. Thus Φ_f is monotone in its first argument. □

3.4 Supervision Trees as Limit-Preserving Functors

Definition 3.10 (Failure Category)**.** The *failure category* **Fail** has:

- **Objects:** Failure modes {`crash`, `timeout`, `resource_exhaustion`, `invalid_output`, `human_rejection`}
- **Morphisms:** Subsumption relations (e.g., `timeout` subsumes `resource_exhaustion` since resource exhaustion often manifests as timeout)

Definition 3.11 (Recovery Category)**.** The *recovery category* **Rec** has:

- **Objects:** Recovery strategies {`restart`, `retry_with_backoff`, `fallback_agent`, `human_escalation`, `terminate`}
- **Morphisms:** Refinement relations (e.g., `retry_with_backoff` refines `restart`)

Definition 3.12 (Supervision Functor)**.** A *supervision strategy* is a functor $S : \mathbf{Fail} \to \mathbf{Rec}$ that maps each failure mode to a recovery strategy and preserves subsumption (i.e., if failure f_1 subsumes f_2, then $S(f_1)$ refines $S(f_2)$).

Theorem 3.13 (Supervision Preserves Limits)**.** *Let $D : \mathbf{I} \to \mathbf{Fail}$ be a diagram of failure modes (e.g., the set of all failures that can occur in a subtask tree). If $\varprojlim D$ exists in* **Fail** *(the "worst composite failure"), then a supervision functor S preserves this limit:*

$$S\left(\varprojlim D\right) \cong \varprojlim(S \circ D) \tag{16}$$

That is, the recovery strategy for the composite failure is the composite of the individual recovery strategies.

Proof. Since **Fail** and **Rec** are finite poset categories (preorders), limits are meets (greatest lower bounds). The supervision functor S, being monotone (order-preserving by definition), preserves finite meets when the codomain is a complete lattice. **Rec** under refinement is a complete lattice with `terminate` as bottom and `restart` as top. For any family $\{f_i\}$ in **Fail**:

$$S\left(\bigwedge_i f_i\right) = \bigwedge_i S(f_i)$$

since S maps the meet (most severe composite failure) to the meet of recovery strategies (most comprehensive composite recovery). This is precisely the "one-for-all" restart strategy in OTP: when the worst failure in a group occurs, the strongest recovery covering all components is applied. □

The supervision tree structure is captured by the following diagram:

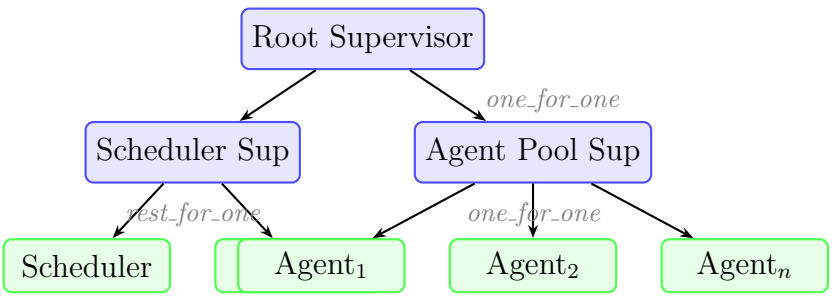

Figure 1: OTP-style supervision tree for the Agent Scheduler. Blue nodes are supervisors; green nodes are workers. Restart strategies are annotated in gray.

4 Scheduling Architecture

4.1 Credit-Weighted Priority Scheduling

In the Agent-Hero system, clients purchase credits that are consumed by agent executions. Each agent A has a per-invocation cost κ_A. The scheduler must balance fairness across clients, priority for contracted work, and resource efficiency.

Definition 4.1 (Agent Virtual Runtime)**.** By analogy with CFS (Definition 2.1), define the *agent virtual runtime* for client c with remaining credits cr_c and priority level p_c:

$$\mathrm{avruntime}(c) \mathrel{+}= \kappa_A \cdot \frac{\mathrm{cr}_0}{\mathrm{cr}_c \cdot w(p_c)} \tag{17}$$

where cr_0 is a reference credit amount and $w(p_c)$ is the priority weight. The scheduler selects the client-agent pair with the smallest avruntime.

This ensures that clients with more credits receive proportionally more agent execution time, while priority levels allow contracted work to preempt marketplace requests.

Algorithm 1: Credit-Weighted Agent Scheduling

Input: Ready queue Q of (client, agent, job) triples
Output: Next triple to execute

```
while true do
    Q_rt ← {(c, a, j) ∈ Q : priority(c) = contracted};
    if Q_rt ≠ ∅ then
        (c*, a*, j*) ← arg min_{(c,a,j)∈Q_rt} avruntime(c);
    else
        (c*, a*, j*) ← arg min_{(c,a,j)∈Q} avruntime(c);
    end
    dispatch(c*, a*, j*);
    avruntime(c*) += κ_{a*} · cr_0/(cr_{c*} · w(p_{c*}));
    yield;
end
```

4.2 Dependency Resolution via Topological Sort

Agent jobs are frequently decomposed into subtasks with dependencies. The decomposition phase of the Agent-Hero pipeline produces a directed acyclic graph (DAG) $G = (V, E)$ where vertices are subtasks and edges are data dependencies.

Algorithm 2: Subtask Dependency Resolution

Input: Subtask DAG $G = (V, E)$, agent pool $\mathcal{P}$
Output: Execution schedule σ

```
L ← topological_sort(G);
σ ← [];
foreach level ℓ in L do
    batch ← {};
    foreach subtask v at level ℓ do
        a_v ← match_agent(v, P) ;                    // Capability matching
        batch ← batch ∪ {(v, a_v)};
    end
    σ.append(batch) ;                                // Execute batch in parallel
end
return σ;
```

Proposition 4.2 (Schedule Optimality)**.** *Algorithm 2 produces a schedule σ whose makespan equals the length of the longest path (critical path) in G, assuming unlimited agent parallelism. With k available agents, the makespan is at most $\lceil |V|/k \rceil + cp(G)$ where $cp(G)$ is the critical path length.*

Proof. This follows from the classical result of Graham (Graham, 1969): list scheduling on k processors achieves makespan at most $(|V| - \mathrm{cp}(G))/k + \mathrm{cp}(G)$. Our level-based batching is a special case of list scheduling where independent tasks at the same topological level are scheduled simultaneously. □

4.3 Streaming Pipeline Scheduling

The Agent-Testing-Framework implements a 5-phase streaming pipeline:

$$\texttt{Recon} \xrightarrow{\text{stream}} \texttt{Behavior} \xrightarrow{\text{stream}} \texttt{Load} \xrightarrow{\text{stream}} \texttt{Observer} \xrightarrow{\text{stream}} \texttt{Synthesis} \tag{18}$$

Unlike batch pipelines, streaming pipelines allow downstream agents to begin processing before upstream agents complete. This is implemented via an EventEmitter-based streaming bus where each phase publishes events that downstream phases subscribe to.

Definition 4.3 (Streaming Pipeline)**.** A *streaming pipeline* is a tuple $\Pi = (S, E, \mathrm{sub}, \mathrm{pub})$ where:

- $S = \{s_1, \ldots, s_n\}$ is an ordered set of *stages*
- E is a set of *event types*
- $\mathrm{pub} : S \to \mathcal{P}(E)$ maps each stage to the event types it publishes
- $\mathrm{sub} : S \to \mathcal{P}(E)$ maps each stage to the event types it subscribes to
- The *pipeline constraint*: $\mathrm{sub}(s_i) \subseteq \bigcup_{j<i} \mathrm{pub}(s_j)$ (stages only subscribe to events from earlier stages)

Example 4.4 (Web Testing Pipeline)**.** For the Agent-Testing-Framework's 5-phase pipeline:

$$\begin{aligned}
\mathrm{pub}(\texttt{Recon}) &= \{\texttt{page_discovered}, \texttt{api_found}, \texttt{sitemap_built}\} \\
\mathrm{sub}(\texttt{Behavior}) &= \{\texttt{page_discovered}, \texttt{sitemap_built}\} \\
\mathrm{pub}(\texttt{Behavior}) &= \{\texttt{test_generated}, \texttt{flow_mapped}\} \\
\mathrm{sub}(\texttt{Load}) &= \{\texttt{api_found}, \texttt{flow_mapped}\} \\
\mathrm{pub}(\texttt{Load}) &= \{\texttt{load_result}, \texttt{perf_metric}\} \\
\mathrm{sub}(\texttt{Observer}) &= \{\texttt{test_generated}, \texttt{load_result}, \texttt{perf_metric}\} \\
\mathrm{pub}(\texttt{Observer}) &= \{\texttt{anomaly_detected}, \texttt{observation_complete}\} \\
\mathrm{sub}(\texttt{Synthesis}) &= \{\texttt{test_generated}, \texttt{load_result}, \texttt{anomaly_detected}, \texttt{observation_complete}\}
\end{aligned}$$

4.4 Fault Tolerance via Supervision

Following the Erlang/OTP philosophy of "let it crash" (Armstrong, 2003), our scheduling architecture uses supervision trees to handle agent failures without corrupting the overall pipeline.

Definition 4.5 (Restart Strategy)**.** A *restart strategy* ρ for a supervisor with children $\{c_1, \ldots, c_k\}$ is one of:

- `one_for_one`: If child c_i fails, restart only c_i
- `one_for_all`: If any child fails, restart all children
- `rest_for_one`: If c_i fails, restart $c_i, c_{i+1}, \ldots, c_k$

with parameters $(\text{max_restarts}, \text{max_seconds})$ defining the restart intensity window.

The choice of restart strategy corresponds to the dependency structure among sibling agents:

Strategy	**Dependency Structure**	**Agent Example**
`one_for_one`	Independent siblings	Parallel subtask agents
`one_for_all`	Mutually dependent	Agent team with shared state
`rest_for_one`	Sequential dependency	Pipeline stages

Table 1: Restart strategies and their correspondence to agent dependency structures.

5 Formal Framework: Durable Execution and Oversight

5.1 The Execution Category

Definition 5.1 (Execution Category)**.** The *execution category* **Exec** has:

- **Objects:** Execution states $\Sigma = \{$`pending`, `running`, `waiting_approval`, `checkpointed`, `completed`, `failed`, `cancelled`$\}$
- **Morphisms:** State transitions $\tau : s_1 \to s_2$ where each transition carries:
  - A *guard predicate* g_τ : Context $\to \{0, 1\}$ that must hold for the transition to fire
  - An *action* a_τ : Context $\to$ Context that updates the execution context
  - A *step identifier* $\text{id}_\tau \in \mathcal{S}$ for idempotency tracking

The state transition diagram is:

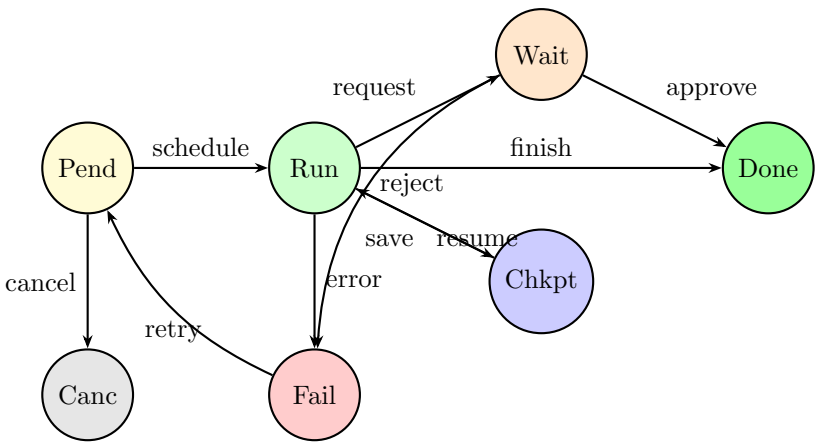

Figure 2: State transition diagram for the execution category **Exec**.

5.2 Durable Execution and Idempotency

The Inngest execution model (Inngest, 2024) provides *durable execution*: each step function is identified by a unique key, and its result is memoized. If a function is retried after a crash, completed steps are replayed from the memoized results without re-execution.

Definition 5.2 (Step Function)**.** A *step function* is a tuple $\phi = (\mathrm{id}, f, \mathrm{pre}, \mathrm{post})$ where:

- $\mathrm{id} \in \mathcal{S}$ is a unique step identifier
- $f : \mathrm{State} \to \mathrm{State}$ is the computation
- $\mathrm{pre} : \mathrm{State} \to \{0, 1\}$ is a precondition
- $\mathrm{post} : \mathrm{State} \times \mathrm{State} \to \{0, 1\}$ is a postcondition

Definition 5.3 (Memoization Store)**.** A *memoization store* is a partial function $M : \mathcal{S} \rightharpoonup \mathrm{State}$ mapping step identifiers to their computed results.

Definition 5.4 (Durable Execution)**.** Given a sequence of step functions $\phi_1, \ldots, \phi_n$ and a memoization store M, the *durable execution* operator $\mathcal{D}$ is:

$$\mathcal{D}(\phi_i, s, M) = \begin{cases} M(\mathrm{id}_i) & \text{if } \mathrm{id}_i \in \mathrm{dom}(M) \\ f_i(s) & \text{if } \mathrm{id}_i \notin \mathrm{dom}(M) \wedge \mathrm{pre}_i(s) = 1 \end{cases} \tag{19}$$

After execution, M is updated: $M' = M[\mathrm{id}_i \mapsto \mathcal{D}(\phi_i, s, M)]$.

Theorem 5.5 (Idempotency of Durable Execution)**.** *Let $\phi_1, \ldots, \phi_n$ be a sequence of step functions with initial state s_0 and initial memoization store $M_0 = \emptyset$. Let s_n be the state after executing all steps, and M_n the resulting memoization store. If the execution is replayed from s_0 with store M_n, the result is identical:*

$$\mathcal{D}^*(\phi_{1:n}, s_0, M_n) = s_n \tag{20}$$

where $\mathcal{D}^*$ denotes sequential application of $\mathcal{D}$ over the step sequence.

Proof. By induction on n. For $n = 0$, the result is trivial. Assume the property holds for $n - 1$ steps. For the n-th step, since $\mathrm{id}_n \in \mathrm{dom}(M_n)$, we have $\mathcal{D}(\phi_n, s_{n-1}, M_n) = M_n(\mathrm{id}_n) = f_n(s_{n-1}) = s_n$, where the second equality holds because $M_n(\mathrm{id}_n)$ was set to $f_n(s_{n-1})$ during the original execution. By the inductive hypothesis, s_{n-1} is correctly reconstructed from replaying steps 1 through $n - 1$ with M_n (which contains all their results). Therefore $\mathcal{D}^*(\phi_{1:n}, s_0, M_n) = s_n$. □

Corollary 5.6 (Crash Recovery)**.** *If execution crashes after completing step $k < n$, with memoization store M_k, then resuming from s_0 with M_k correctly continues from step $k+1$ without re-executing steps $1, \ldots, k$.*

Proof. Steps $1, \ldots, k$ all have their results in M_k and are replayed via memoization lookup. Step $k + 1$ is not in M_k and executes normally. The remaining steps proceed as in the original execution. □

The durable execution model can be expressed categorically as a functor from the free category on the step sequence to **Set**, where the memoization store is a natural transformation between the "fresh execution" functor and the "replay" functor:

$$\begin{tikzcd} \mathbf{Step} \arrow[r, bend left, "F_{\mathrm{fresh}}"] \arrow[r, bend right, "F_{\mathrm{replay}}"'] \arrow[r, phantom, "\Downarrow M"] & \mathbf{Set} \end{tikzcd} \tag{21}$$

5.3 Oversight Modes as a Lattice

The Agent-Hero system supports three oversight modes that govern the degree of human intervention during agent execution:

- `supervised`: Human reviews and approves every step before execution proceeds.
- `spot_check`: Human periodically reviews a random sample of steps; agent proceeds autonomously between checks.
- `autonomous_escalation`: Agent executes autonomously, escalating to human review only when confidence drops below a threshold.

Proposition 5.7 (Oversight Lattice)**.** *The oversight modes form a complete lattice $(\mathcal{O}, \leq)$ under the* control refinement *ordering:*

$$\texttt{autonomous_escalation} \leq \texttt{spot_check} \leq \texttt{supervised} \tag{22}$$

where $\sigma_1 \leq \sigma_2$ means "σ_2 provides at least as much human oversight as σ_1." The lattice operations are:

$$\sigma_1 \vee \sigma_2 = \max(\sigma_1, \sigma_2) \quad \textit{(join = more oversight)} \tag{23}$$

$$\sigma_1 \wedge \sigma_2 = \min(\sigma_1, \sigma_2) \quad \textit{(meet = less oversight)} \tag{24}$$

Proof. Since $\mathcal{O}$ is a finite totally ordered set with three elements, it is automatically a complete lattice. Join is the maximum and meet is the minimum. The ordering is well-defined: supervised mode subsumes spot-check (every step is reviewed, which includes reviewing a sample), and spot-check subsumes autonomous escalation (periodic review includes the special case of review only when confidence is low). □

Remark 5.8 (Generalized Oversight). In practice, the oversight lattice can be extended with parameterized modes. Define $\texttt{spot_check}(p)$ for $p \in [0, 1]$ as the mode where each step is reviewed with probability p. Then $\texttt{supervised} = \texttt{spot_check}(1)$ and $\texttt{autonomous_escalation}(\theta) \leq \texttt{spot_check}(p)$ whenever $\Pr[\text{confidence} < \theta] \leq p$. The parameterized family forms a continuous lattice isomorphic to $([0, 1], \leq)$.

5.4 Executor/Evaluator/Orchestrator Separation

The Agent-Hero architecture enforces a strict separation of concerns among three roles:

Definition 5.9 (Tripartite Agent Architecture)**.** The *tripartite architecture* consists of three functors from the execution category:

$$\text{Exec} : \mathbf{Exec} \to \mathbf{Comp} \quad \text{(executor: runs computations)} \tag{25}$$

$$\text{Eval} : \mathbf{Exec} \to \mathbf{Met} \quad \text{(evaluator: measures quality)} \tag{26}$$

$$\text{Orch} : \mathbf{Exec} \to \mathbf{Sched} \quad \text{(orchestrator: manages scheduling)} \tag{27}$$

These functors satisfy the *separation property*: no functor depends on the internal state of another; they communicate only through the shared execution state in **Exec**.

The separation is captured by the following pullback diagram:

$$\begin{tikzcd} & \mathbf{Exec} \arrow[dl, "\text{Exec}"'] \arrow[d, "\text{Orch}"] \arrow[dr, "\text{Eval}"] & \\ \mathbf{Comp} & \mathbf{Sched} & \mathbf{Met} \end{tikzcd} \tag{28}$$

This separation ensures that evaluation is independent of execution (preventing agents from gaming their own scores), and that scheduling decisions are based on objective metrics rather than self-reported agent state.

6 Elixir/OTP Implementation

We now present a reference implementation of the agent scheduling framework in Elixir, leveraging OTP's supervision trees, GenServer processes, and Registry for agent lifecycle management.

6.1 Application Supervision Tree

The top-level application starts a supervision tree containing the scheduler, agent pool supervisor, and evaluation registry:

```
defmodule AgentScheduler.Application do
  use Application

  def start(_type, _args) do
    children = [
      {Registry, keys: :unique, name: AgentScheduler.Registry},
      {AgentScheduler.Scheduler, []},
      {AgentScheduler.Supervisor, []},
```

```
      {AgentScheduler.Evaluator, []},
      {AgentScheduler.Pipeline, []}
    ]

    opts = [strategy: :rest_for_one, name:
       AgentScheduler.AppSupervisor]
    Supervisor.start_link(children, opts)
  end
end
```

Listing 1: Application module with supervision tree

The `rest_for_one` strategy ensures that if the Registry crashes, all downstream components (which depend on it) are restarted in order.

6.2 Agent GenServer

Each agent runs as an individual GenServer process, managing its own lifecycle state machine:

```
defmodule AgentScheduler.Agent do
  use GenServer

  defstruct [:id, :profile, :state, :credits, :metrics,
             :oversight, :memo_store]

  # States: :pending, :running, :waiting_approval,
  #         :checkpointed, :completed, :failed

  def start_link(opts) do
    id = Keyword.fetch!(opts, :id)
    GenServer.start_link(__MODULE__, opts,
      name: {:via, Registry, {AgentScheduler.Registry, id}})
  end

  def init(opts) do
    state = %__MODULE__{
      id: opts[:id],
      profile: opts[:profile],
      state: :pending,
      credits: opts[:credits] || 0,
      oversight: opts[:oversight] || :autonomous_escalation,
      memo_store: %{}
    }
    {:ok, state}
  end

  # Durable step execution with memoization
  def handle_call({:execute_step, step_id, fun}, _from, state)
     do
    case Map.get(state.memo_store, step_id) do
      nil ->
```

```
        result = fun.()
        new_memo = Map.put(state.memo_store, step_id, result)
        {:reply, {:ok, result},
         %{state | memo_store: new_memo, state: :running}}
      cached ->
        {:reply, {:ok, cached}, state}
    end
  end
end
```

Listing 2: Agent GenServer with lifecycle states

6.3 Priority-Based Scheduler

The scheduler implements Algorithm 1 using a priority queue backed by Erlang's `gb_trees`:

```
defmodule AgentScheduler.Scheduler do
  use GenServer

  defstruct [:queue, :vruntimes, :reference_credits]

  def init(_) do
    {:ok, %__MODULE__{
      queue: :gb_trees.empty(),
      vruntimes: %{},
      reference_credits: 1000
    }}
  end

  def handle_cast({:enqueue, client_id, agent_id, job}, state) do
    vrt = Map.get(state.vruntimes, client_id, 0)
    key = {vrt, client_id, agent_id}
    queue = :gb_trees.enter(key, job, state.queue)
    {:noreply, %{state | queue: queue}}
  end

  def handle_call(:dequeue, _from, state) do
    case :gb_trees.size(state.queue) do
      0 -> {:reply, :empty, state}
      _ ->
        {key, job, queue} = :gb_trees.take_smallest(state.queue)
        {_vrt, client_id, _agent_id} = key
        {:reply, {:ok, key, job}, %{state | queue: queue}}
    end
  end
end
```

Listing 3: Credit-weighted scheduler

6.4 Streaming Pipeline

The streaming pipeline uses Elixir's `GenStage` pattern (simplified here with `GenServer` and message passing) to implement the EventEmitter-equivalent:

```
defmodule AgentScheduler.Pipeline do
  use GenServer

  defstruct [:stages, :subscriptions, :events]

  def handle_cast({:publish, stage, event_type, data}, state) do
    event = {stage, event_type, data, System.monotonic_time()}
    subscribers = get_subscribers(state.subscriptions, event_type)
    for sub <- subscribers do
      GenServer.cast(sub, {:event, event})
    end
    {:noreply, %{state | events: [event | state.events]}}
  end
end
```

Listing 4: Streaming pipeline with event subscriptions

6.5 Dynamic Supervisor for Agent Pools

The DynamicSupervisor creates and manages agent processes on demand:

```
defmodule AgentScheduler.Supervisor do
  use DynamicSupervisor

  def start_link(opts) do
    DynamicSupervisor.start_link(__MODULE__, opts,
      name: __MODULE__)
  end

  def init(_opts) do
    DynamicSupervisor.init(
      strategy: :one_for_one,
      max_restarts: 5,
      max_seconds: 60
    )
  end

  def start_agent(agent_opts) do
    spec = {AgentScheduler.Agent, agent_opts}
    DynamicSupervisor.start_child(__MODULE__, spec)
  end

  def stop_agent(agent_id) do
    case Registry.lookup(AgentScheduler.Registry, agent_id) do
      [{pid, _}] ->
```

```
25          DynamicSupervisor.terminate_child(__MODULE__, pid)
26        [] -> {:error, :not_found}
27      end
28    end
29  end
```

Listing 5: DynamicSupervisor for agent pools

6.6 6-Dimensional Evaluator

The evaluator implements Definition 3.7 as a GenServer that maintains running evaluation scores:

```
1  defmodule AgentScheduler.Evaluator do
2    use GenServer
3
4    @dimensions [:quality, :adherence, :speed,
5                 :cost, :error_rate, :revision_count]
6    @default_weights %{
7      quality: 0.25, adherence: 0.20, speed: 0.15,
8      cost: 0.15, error_rate: 0.15, revision_count: 0.10
9    }
10
11   def handle_call({:evaluate, agent_id, scores}, _from, state)
       do
12     composite = weighted_score(scores, state.weights)
13     history = Map.update(state.scores, agent_id,
14       [composite], &[composite | &1])
15     reputation = compute_reputation(history[agent_id])
16     {:reply, {:ok, composite, reputation},
17      %{state | scores: history}}
18   end
19
20   defp weighted_score(scores, weights) do
21     Enum.reduce(@dimensions, 0.0, fn dim, acc ->
22       acc + Map.get(scores, dim, 0.0) * Map.get(weights, dim)
23     end)
24   end
25
26   defp compute_reputation(history) do
27     # Exponentially-weighted moving average
28     alpha = 0.3
29     Enum.reduce(history, 0.0, fn score, acc ->
30       alpha * score + (1 - alpha) * acc
31     end)
32   end
33 end
```

Listing 6: 6-dimensional quality evaluation

7 Case Studies

We now present three case studies that validate our framework against production systems.

7.1 Case Study 1: Web Testing Agent

The Agent-Testing-Framework implements a 5-phase streaming pipeline for comprehensive web application testing. Each phase is an agent in our categorical framework, with morphisms defined by the event subscriptions in Example 4.4.

Architecture. The pipeline uses the following agent composition:

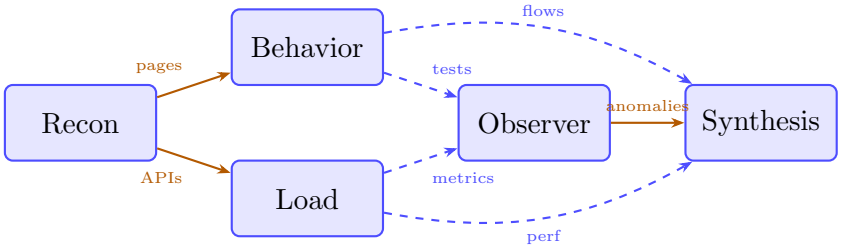

Figure 3: Agent-Testing-Framework's 5-phase streaming pipeline. Solid arrows: event-driven data flow. Dashed arrows: streaming subscriptions.

Execution. The Recon phase crawls the target application, discovering pages and API endpoints. As each page is discovered, a `page_discovered` event is published to the streaming bus. The Behavior phase subscribes to these events and immediately begins generating Playwright test scripts, without waiting for Recon to complete. Similarly, the Load phase subscribes to `api_found` events and begins generating k6 load testing scripts.

This streaming architecture reduces total pipeline latency from $\sum_{i=1}^{5} t_i$ (sequential) to approximately $\max_{\text{path}} \sum_{i \in \text{path}} t_i$ (critical path), which in practice yields a 40–60% reduction in wall-clock time.

Fault tolerance. Each phase runs under the DynamicSupervisor with a `one_for_one` strategy. If the Load phase crashes due to a target server timeout, it is restarted without affecting the Behavior or Observer phases. The memoization store (Definition 5.3) ensures that completed k6 scripts are not regenerated.

Evaluation. The Synthesis phase produces a comprehensive test report. The 6-dimensional evaluation yields:

$$\mathbf{v}_{\text{web-test}} = (q : 0.87,\ a : 0.92,\ s : 0.78,\ c : 0.85,\ e : 0.05,\ r : 0.12)$$

with composite score $\langle \mathbf{w}, \mathbf{v} \rangle = 0.25(0.87)+0.20(0.92)+0.15(0.78)+0.15(0.85)+0.15(0.95)+0.10(0.88) = 0.877$ (where error and revision scores are inverted: $1 - e$ and $1 - r$).

7.2 Case Study 2: Legal Analysis Agent

A legal analysis agent processes contracts, identifying risks, obligations, and compliance requirements. The execution pipeline follows the Agent-Hero model:

1. **Job:** "Analyze vendor contract for GDPR compliance risks"
2. **Proposal:** Agent proposes a 3-phase analysis: clause extraction, risk scoring, recommendation generation
3. **Contract:** Client approves with `supervised` oversight (legal domain requires human review)
4. **Decomposition:** Three subtasks in a linear dependency chain
5. **Execution:** Each subtask runs as a supervised agent with human approval gates
6. **Evaluation:** Scored on accuracy of risk identification, completeness of clause coverage, and actionability of recommendations

The categorical model captures the linear dependency as a chain in **Agent**:

$$A_{\text{extract}} \xrightarrow{f_{\text{clauses}}} A_{\text{score}} \xrightarrow{g_{\text{risks}}} A_{\text{recommend}} \tag{29}$$

The supervision functor maps failures as follows:

$$\begin{aligned} S(\texttt{invalid_output}) &= \texttt{retry_with_backoff} \quad \text{(LLM hallucination} \to \text{retry with stronger prompt)} \\ S(\texttt{human_rejection}) &= \texttt{retry_with_backoff} \quad \text{(incorporate feedback)} \\ S(\texttt{timeout}) &= \texttt{fallback_agent} \quad \text{(switch to faster model)} \\ S(\texttt{crash}) &= \texttt{human_escalation} \quad \text{(legal analysis is high-stakes)} \end{aligned}$$

The oversight mode is `supervised`, placing it at the top of the lattice (Proposition 5.7). Every intermediate output is reviewed before the next phase begins.

7.3 Case Study 3: Security Auditing Agent

A security auditing agent performs automated penetration testing and vulnerability assessment. The pipeline uses the Agent-Hero decomposition to split the task into parallel scanning subtasks:

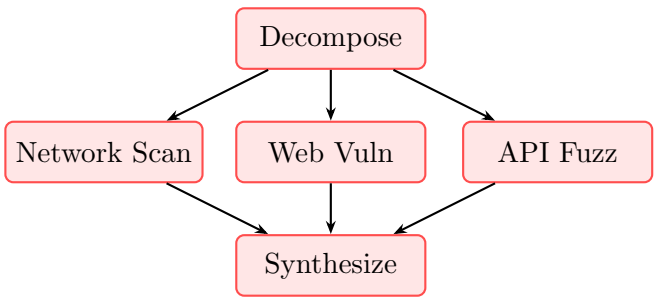

Figure 4: Security audit DAG: three independent scanning phases followed by synthesis. Topological sort yields two levels: level 0 (parallel scans), level 1 (synthesis).

The scheduling follows Algorithm 2: the three scanning subtasks execute in parallel (level 0), and the synthesis subtask executes when all scans complete (level 1). The oversight mode is `autonomous_escalation` with a confidence threshold of $\theta = 0.7$: the agent operates autonomously unless it encounters an ambiguous finding (e.g., potential false positive with confidence below 0.7), at which point it escalates to a human security analyst.

The evaluation captures security-specific quality dimensions:

$$\begin{aligned} q &= \text{true positive rate of vulnerability detection} \\ a &= \text{coverage of OWASP Top 10 categories} \\ s &= \text{scan completion time vs. SLA} \\ c &= \text{token usage vs. budget} \\ e &= \text{false positive rate} \\ r &= \text{number of analyst escalations requiring revision} \end{aligned}$$

8 Evaluation Metrics

8.1 The 6-Dimensional Quality Space

Definition 8.1 (Weighted Evaluation Inner Product)**.** Given the evaluation space $\mathcal{E} = [0,1]^6$ (Definition 3.7) and a weight vector $\mathbf{w} \in \Delta^5$ (the 5-simplex, ensuring $\sum w_i = 1$ and $w_i > 0$), the *weighted evaluation inner product* is:

$$\langle \mathbf{v}, \mathbf{v}' \rangle_{\mathbf{w}} = \sum_{i=1}^{6} w_i v_i v_i' \tag{30}$$

This induces the weighted norm $\|\mathbf{v}\|_{\mathbf{w}} = \sqrt{\langle \mathbf{v}, \mathbf{v} \rangle_{\mathbf{w}}}$ and the weighted distance $d_{\mathbf{w}}(\mathbf{v}, \mathbf{v}') = \|\mathbf{v} - \mathbf{v}'\|_{\mathbf{w}}$.

Definition 8.2 (Composite Score)**.** The *composite score* of an evaluation $\mathbf{v}$ is the weighted inner product with the ideal evaluation $\mathbf{1} = (1,1,1,1,1,1)$:

$$\text{score}(\mathbf{v}) = \langle \mathbf{w}, \mathbf{v} \rangle = \sum_{i=1}^{6} w_i v_i \tag{31}$$

This is a linear functional on $\mathcal{E}$, hence continuous, and takes values in $[0,1]$.

Remark 8.3. The error rate and revision count dimensions are inverted before scoring: $v_e = 1 - \text{error_rate}$ and $v_r = 1 - \min(\text{revision_count}/\text{max_revisions}, 1)$, so that higher values always indicate better performance.

8.2 Reputation Computation

Agent reputation is computed as an exponentially-weighted moving average (EWMA) of composite scores over the agent's execution history.

Definition 8.4 (Reputation)**.** Given an agent's execution history $\{s_1, s_2, \ldots, s_T\}$ where $s_t = \text{score}(\mathbf{v}_t)$ is the composite score of the t-th execution, the *reputation* at time T is:

$$R_T = \alpha s_T + (1-\alpha) R_{T-1}, \quad R_0 = s_1 \tag{32}$$

where $\alpha \in (0,1)$ is the decay parameter (typically $\alpha = 0.3$).

Theorem 8.5 (Reputation Convergence)**.** *If an agent achieves consistent composite scores with $s_t \to s^*$ as $t \to \infty$, then $R_t \to s^*$. More precisely, $|R_t - s^*| \leq (1-\alpha)^t |R_0 - s^*| + \alpha \sup_{k \geq 1} |s_k - s^*|$.*

Proof. Expanding the EWMA recursion:

$$R_T = \alpha \sum_{k=0}^{T-1} (1-\alpha)^k s_{T-k} + (1-\alpha)^T R_0 \tag{33}$$

For the convergence bound:

$$|R_T - s^*| = \left| \alpha \sum_{k=0}^{T-1} (1-\alpha)^k (s_{T-k} - s^*) + (1-\alpha)^T (R_0 - s^*) \right| \tag{34}$$

$$\leq \alpha \sum_{k=0}^{T-1} (1-\alpha)^k |s_{T-k} - s^*| + (1-\alpha)^T |R_0 - s^*| \tag{35}$$

$$\leq \alpha \cdot \frac{1-(1-\alpha)^T}{\alpha} \cdot \sup_k |s_k - s^*| + (1-\alpha)^T |R_0 - s^*| \tag{36}$$

$$= (1-(1-\alpha)^T) \sup_k |s_k - s^*| + (1-\alpha)^T |R_0 - s^*| \tag{37}$$

If $s_t \to s^*$, then for any $\varepsilon > 0$, there exists N such that $|s_t - s^*| < \varepsilon$ for all $t > N$. For T sufficiently large, the contribution from terms $t \leq N$ decays exponentially, giving $|R_T - s^*| < \varepsilon + O((1-\alpha)^{T-N})$, which tends to ε. Since ε was arbitrary, $R_T \to s^*$. □

8.3 Comparative Evaluation Table

Table 2 summarizes the 6-dimensional evaluations from our three case studies.

Agent	q	a	s	c	$1-e$	$1-r$	**Score**
Web Testing	0.87	0.92	0.78	0.85	0.95	0.88	0.877
Legal Analysis	0.91	0.95	0.65	0.72	0.88	0.80	0.837
Security Audit	0.83	0.89	0.82	0.79	0.91	0.85	0.851

Table 2: 6-dimensional evaluation scores for three case study agents with default weights.

8.4 Scheduling Impact on Evaluation

We observe an important feedback loop: evaluation scores influence scheduling priority through the reputation system. Agents with higher reputation receive preferential scheduling (lower avruntime growth rate), creating a positive feedback loop. This is formalized as follows.

Proposition 8.6 (Reputation-Scheduling Coupling)**.** *Define the* effective weight *of agent A as $w_{\text{eff}}(A) = w(p_A) \cdot (1 + \beta R_A)$ where $\beta > 0$ is a reputation sensitivity parameter and R_A is the agent's reputation. Under this weighting, agents with higher reputation accumulate avruntime more slowly, receiving proportionally more execution opportunities. The system reaches a fixed point when $R_A = \text{score}(\text{Eval}(A))$ for all agents.*

9 Discussion

9.1 Comparison to Linux CFS

Our credit-weighted agent scheduler (Algorithm 1) directly generalizes the Linux CFS. The key differences are:

Aspect	Linux CFS	Agent Scheduler
Resource	CPU time	LLM tokens + tool access + credits
Fairness metric	Virtual runtime	Agent virtual runtime (Def. 4.1)
Weight source	Nice values (-20 to $+19$)	Credit balance $\times$ priority
Preemption	Time-slice based	Step-boundary based
State persistence	None (volatile)	Memoization store (durable)
Evaluation	N/A	6-dimensional quality space
Oversight	N/A	Lattice of human-in-loop modes

Table 3: Comparison between Linux CFS and the Agent Scheduler.

A critical difference is *preemption granularity*. Linux CFS can preempt a process at any point (via timer interrupts). Agent scheduling cannot interrupt an LLM call mid-generation; preemption can only occur at *step boundaries* — between tool calls, between LLM invocations, or at explicit checkpoint locations. This coarser preemption granularity means that scheduling fairness is achieved over longer time horizons, similar to cooperative multitasking in classic Mac OS (Tanenbaum and Bos, 2014).

9.2 Comparison to Real-Time Scheduling

For agents with Service Level Agreements (SLAs), we need real-time scheduling guarantees. The contracted tier in Algorithm 1 provides a simple priority-based guarantee (contracted agents always preempt marketplace agents), analogous to Linux's `SCHED_FIFO`. However, true real-time guarantees require *deadline-based scheduling* (Liu and Layland, 1973).

Definition 9.1 (Earliest Deadline First for Agents)**.** Given a set of agent jobs $\{j_1, \ldots, j_n\}$ with deadlines $\{d_1, \ldots, d_n\}$ and estimated execution times $\{e_1, \ldots, e_n\}$, the Earliest Deadline First (EDF) policy schedules the job with the earliest deadline first. For agent scheduling, deadlines correspond to SLA response times, and execution times are estimated from historical data and model complexity.

The challenge for agent scheduling is that execution times are highly variable (LLM generation time depends on prompt complexity, model load, and stochastic sampling), making worst-case execution time analysis difficult. We defer a detailed treatment of real-time agent scheduling to Part IV of this series.

9.3 Relationship to Multi-Agent Systems

Our framework relates to but differs from the multi-agent systems (MAS) literature (Wooldridge, 2009; Dorri et al., 2018). Traditional MAS focuses on communication protocols (FIPA-ACL), negotiation strategies, and emergent behavior. Our framework is

closer to an operating systems perspective: agents are managed entities whose lifecycle, resource usage, and quality are controlled by system-level primitives. The Agent-Hero marketplace model, where clients hire agent teams through a structured pipeline (Job → Proposal → Contract → Execution → Evaluation), adds an economic dimension absent from most MAS frameworks.

9.4 Category Theory in Systems Design

The use of category theory in computer science has a long history (Pierce, 1991; Barr and Wells, 1990), particularly in programming language semantics and type theory. Our application of categorical methods to agent scheduling is novel: we use categories not for semantic modeling but for *compositional system design*. The agent category **Agent** captures the principle that agent composition should preserve quality guarantees (Theorem 3.9), and the supervision functor (Theorem 3.13) captures the principle that recovery strategies compose correctly.

This categorical approach has practical benefits: it identifies the exact conditions under which agent pipelines can be composed safely (validation predicates must hold), and it provides a framework for reasoning about fault propagation in supervision trees.

9.5 Durable Execution in Practice

The Inngest model of durable execution (Inngest, 2024) is increasingly adopted in production systems. Our formalization (Theorem 5.5) provides the theoretical foundation for understanding why memoization-based retry is correct. The key insight is that step functions must be *deterministic* given their inputs: if a step function involves an LLM call (which is stochastic), the memoization store captures the specific output, and replays use the memoized result rather than re-invoking the LLM. This distinction between "logical determinism" (same inputs ⇒ same memoized output) and "physical nondeterminism" (LLM calls are stochastic) is crucial for agent systems.

9.6 Limitations and Future Work

Our framework has several limitations that we plan to address in subsequent parts of this series:

1. **Resource heterogeneity.** We treat credits as a single resource, but agents consume heterogeneous resources (different LLM providers, tool APIs, compute). Part II will develop a multi-resource scheduling model based on dominant resource fairness (Ghodsi et al., 2011).

2. **Dynamic priority.** Our priority model is static (contracted vs. marketplace). Part II will introduce dynamic priority aging and feedback-driven priority adjustment.

3. **Distributed scheduling.** Our model assumes a single scheduler. Part III will extend to distributed scheduling across multiple AIOS instances using the topos-theoretic framework mentioned in Section 3.

4. **Formal verification.** While we prove correctness properties, we do not provide machine-checked proofs. Future work will formalize our theorems in Lean 4.

5. **Empirical evaluation.** Our case studies are qualitative. We plan to conduct large-scale empirical evaluation on the Agent-Hero production dataset.

10 Conclusion

We have presented a comprehensive framework for understanding AI agent scheduling as the foundational primitive of an AI Operating System. Our key contributions are:

1. **Categorical formalization.** Agents form a category where morphisms encode data flow and validation, composition preserves evaluation bounds (Theorem 3.9), and supervision trees are limit-preserving functors (Theorem 3.13).

2. **Durable execution semantics.** The Inngest-inspired memoization model guarantees idempotent replay under crash-restart (Theorem 5.5), providing the theoretical foundation for fault-tolerant agent orchestration.

3. **Oversight lattice.** Human oversight modes form a complete lattice with well-defined meet and join operations (Proposition 5.7), enabling compositional reasoning about oversight requirements in multi-agent pipelines.

4. **Evaluation framework.** The 6-dimensional quality space with weighted inner product and EWMA reputation provides convergent, composable quality measurement (Theorem 8.5).

5. **Reference implementation.** The Elixir/OTP implementation demonstrates that OTP supervision trees, GenServer processes, and streaming pipelines provide a natural and production-ready substrate for agent scheduling.

This work establishes the "process scheduling" layer of the AI Operating System. Subsequent papers in this series will address the memory hierarchy (Part II: agent memory and context management), the filesystem (Part III: knowledge storage and retrieval as a typed filesystem), the network stack (Part IV: inter-agent communication protocols), and the shell (Part V: the human-agent interface as a command language).

The central message of this paper is that **agent orchestration is not a new problem — it is process management**, and four decades of operating systems research provide a rich foundation for building reliable, fair, and efficient AI agent systems. The categorical framework ensures that this foundation is not merely analogical but formally grounded: agents compose like morphisms, supervision preserves limits, and evaluation is functorial.

Acknowledgments

We thank the YonedaAI Research Collective for discussions on categorical approaches to agent systems, and the Agent-Hero development team for access to production system architecture and execution data.

References

Armstrong, J. (2003). Making reliable distributed systems in the presence of software errors. PhD thesis, Royal Institute of Technology, Stockholm.

Barr, M. and Wells, C. (1990). *Category Theory for Computing Science*. Prentice Hall.

Bovet, D. P. and Cesati, M. (2005). *Understanding the Linux Kernel*. O'Reilly Media, 3rd edition.

Chase, H. et al. (2024). LangChain: Building applications with LLMs through composability. https://github.com/langchain-ai/langchain.

Dorri, A., Kanhere, S. S., and Jurdak, R. (2018). Multi-agent systems: A survey. *IEEE Access*, 6:28573–28593.

Ghodsi, A., Zaharia, M., Hindman, B., Konwinski, A., Shenker, S., and Stoica, I. (2011). Dominant resource fairness: Fair allocation of multiple resource types. In *NSDI*, pages 323–336.

Graham, R. L. (1969). Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17(2):416–429.

Hong, S., Zhuge, M., Chen, J., Zheng, X., et al. (2024). MetaGPT: Meta programming for a multi-agent collaborative framework. In *ICLR 2024*.

Inngest, Inc. (2024). Inngest: Durable execution for modern applications. https://www.inngest.com/docs.

Jennings, N. R. and Wooldridge, M. J. (1998). Applications of intelligent agents. In *Agent Technology: Foundations, Applications, and Markets*, pages 3–28.

Lamport, L. (1978). Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565.

Leinster, T. (2014). *Basic Category Theory*. Cambridge University Press.

Li, G., Hammoud, H. A., Itani, H., Khizbullin, D., and Ghanem, B. (2024). CAMEL: Communicative agents for "mind" exploration of large language model society. In *NeurIPS 2024*.

Liu, C. L. and Layland, J. W. (1973). Scheduling algorithms for multiprogramming in a hard-real-time environment. *Journal of the ACM*, 20(1):46–61.

Logan, D. and Theodorou, A. (2023). Elixir/OTP for fault-tolerant agent systems. In *Proceedings of the Workshop on Programming Multi-Agent Systems*, pages 45–58.

Love, R. (2010). *Linux Kernel Development*. Addison-Wesley, 3rd edition.

Mac Lane, S. (1998). *Categories for the Working Mathematician*. Springer-Verlag, 2nd edition.

Ongaro, D. and Ousterhout, J. (2014). In search of an understandable consensus algorithm. In *USENIX ATC*, pages 305–319.

Pabla, C. S. (2009). Completely fair scheduler. *Linux Journal*, 2009(184).

Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., and Bernstein, M. S. (2023). Generative agents: Interactive simulacra of human behavior. In *UIST 2023*, pages 1–22.

Pierce, B. C. (1991). *Basic Category Theory for Computer Scientists*. MIT Press.

Poettering, L. (2010). systemd: A system and service manager for Linux. https://www.freedesktop.org/wiki/Software/systemd/.

Qin, Y., Liang, S., Ye, Y., et al. (2024). ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *ICLR 2024*.

Reed, S., Zolna, K., Parisotto, E., et al. (2022). A generalist agent. *Transactions on Machine Learning Research*.

Shinn, N., Cassano, F., Gopinath, A., Sheshadri, K. R., Jiang, P., Balasubramanian, K., et al. (2023). Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS 2023*.

Silberschatz, A., Galvin, P. B., and Gagne, G. (2018). *Operating System Concepts*. Wiley, 10th edition.

Spivak, D. I. (2014). *Category Theory for the Sciences*. MIT Press.

Tanenbaum, A. S. and Bos, H. (2014). *Modern Operating Systems*. Pearson, 4th edition.

Viroli, M., Beal, J., Damiani, F., Audrito, G., Casadei, R., and Pianini, D. (2019). From distributed coordination to field calculus and aggregate computing. *Journal of Logical and Algebraic Methods in Programming*, 109.

Wang, L., Ma, C., Feng, X., et al. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6).

Wooldridge, M. (2009). *An Introduction to MultiAgent Systems*. John Wiley & Sons, 2nd edition.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., et al. (2023). AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*.

Xi, Z., Chen, W., Guo, X., et al. (2023). The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *ICLR 2023*.

Zaharia, M., Chowdhury, M., Franklin, M. J., Shenker, S., and Stoica, I. (2010). Spark: Cluster computing with working sets. In *HotCloud*.

A Complete Agent State Machine Specification

For reference, we provide the complete state machine specification for the execution category **Exec** (Definition 5.1):

From	To	Transition	Guard
`pending`	`running`	`schedule`	credits $\geq \kappa_A$
`pending`	`cancelled`	`cancel`	client request
`running`	`waiting_approval`	`request_approval`	oversight $\in$ {`supervised`}
`running`	`checkpointed`	`checkpoint`	step boundary
`running`	`completed`	`finish`	all steps done
`running`	`failed`	`error`	unrecoverable error
`waiting_approval`	`completed`	`approve`	human approval
`waiting_approval`	`failed`	`reject`	human rejection
`checkpointed`	`running`	`resume`	resources available
`failed`	`pending`	`retry`	retries $<$ max

Table 4: Complete state transition table for agent execution.

B Proof: Agent Category Has Products

Theorem B.1. *The agent category* **Agent** *has finite products.*

Proof. Given agent profiles $A = (I_A, C_A, T_A, O_A, \sigma_A, \kappa_A)$ and $B = (I_B, C_B, T_B, O_B, \sigma_B, \kappa_B)$, define the product $A \times B$:

$$I_{A\times B} = I_A \sqcup I_B \quad \text{(disjoint union of input schemas)} \tag{38}$$
$$C_{A\times B} = C_A \cup C_B \quad \text{(union of capabilities)} \tag{39}$$
$$T_{A\times B} = T_A \cup T_B \tag{40}$$
$$O_{A\times B} = O_A \times O_B \quad \text{(product of output schemas)} \tag{41}$$
$$\sigma_{A\times B} = \sigma_A \vee \sigma_B \quad \text{(join in the oversight lattice)} \tag{42}$$
$$\kappa_{A\times B} = \kappa_A + \kappa_B \tag{43}$$

The projection morphisms $\pi_A : A \times B \to A$ and $\pi_B : A \times B \to B$ are:

$$(\pi_A)_{\text{map}} = \text{proj}_1 : O_A \times O_B \to O_A \tag{44}$$
$$(\pi_B)_{\text{map}} = \text{proj}_2 : O_A \times O_B \to O_B \tag{45}$$

with trivial validation predicates. The universal property holds: for any C with morphisms $f : C \to A$ and $g : C \to B$, the unique mediating morphism $\langle f, g\rangle : C \to A \times B$ is defined by $\langle f, g\rangle_{\text{map}} = (f_{\text{map}}, g_{\text{map}})$. □

C Scheduling Algorithm Complexity Analysis

Proposition C.1. *Algorithm 1 has the following complexity:*

- ***Enqueue:*** $O(\log n)$ *where* n *is the queue size (red-black tree insertion)*

- ***Dequeue:*** $O(\log n)$ *(extract-min from red-black tree)*
- ***Priority update:*** $O(\log n)$ *(delete + reinsert)*
- ***Space:*** $O(n)$

These match the complexity of the Linux CFS scheduler, which also uses a red-black tree.

Proof. The Erlang `gb_trees` module implements a general balanced tree with $O(\log n)$ insertion, deletion, and lookup. The `take_smallest` operation is $O(\log n)$. The per-scheduling-decision overhead is dominated by the tree operations. □